

Работа с Массивом + Строки

std::vector

std::vector – это контейнер из стандартной библиотеки (STL). По своей сути является динамическим массивом, который хранится в памяти и позволяет изменять свой размер во время выполнения программы.

operator=

Копирующий оператор присваивания. Заменяет содержимое одного вектора, копией содержимого другого.

Element Access

- `.at()` – доступ к элементу с проверками границ
- `.front()` – возвращает первый элемент в массиве
- `.back()` – возвращает последний элемент в массиве
- `operator[]` – доступ к элементу без проверки границ
- `.data()` – возвращает указатель на первый элемент массив

Iterators

- `.begin()` – возвращает итератор на начало массива
- `.end()` – возвращает итератор на конец массива

Capacity

- `.empty()` – проверяет пуст ли массив
- `.size()` – возвращает количество элементов в массиве
- `.max_size()` – возвращает максимальное возможное число элементов
- `.reserve()` – резервирует хранилище
- `.capacity()` – возвращает емкость в текущей аллокации

Modifiers

- `.clear()` – очистить массив
- `.insert()` – вставить элементы
- `.insert_range()` – вставить несколько элементов
- `.erase()` – стереть элементы
- `.emplace()` – создает элемент в массиве
- `.push_back()` – добавляет элемент в конец массива
- `.resize()` – изменяет количество элементов в массиве
- `.pop_back()` – извлекает последний элемент

std::string

Для того, чтобы ввести из входного потока строку до первого пробела используется метод `cin`:

```
cin >> s;
```

Если нужно ввести строку, содержащую пробелы, применяется функция `getline`:

```
getline( cin, s );
```

Вывод выполняется стандартным образом:

```
cout << s;
```

Modifiers

<code>clear</code>	clears the contents (public member function)
<code>insert</code>	inserts characters (public member function)
<code>insert_range</code> (C++23)	inserts a range of characters (public member function)
<code>erase</code>	removes characters (public member function)
<code>push_back</code>	appends a character to the end (public member function)
<code>pop_back</code> (DR*)	removes the last character (public member function)
<code>append</code>	appends characters to the end (public member function)
<code>append_range</code> (C++23)	appends a range of characters to the end (public member function)
<code>operator+=</code>	appends characters to the end (public member function)
<code>replace</code>	replaces specified portion of a string (public member function)
<code>replace_with_range</code> (C++23)	replaces specified portion of a string with a range of characters (public member function)
<code>copy</code>	copies characters (public member function)
<code>resize</code>	changes the number of characters stored (public member function)
<code>resize_and_overwrite</code> (C++23)	changes the number of characters stored and possibly overwrites indeterminate contents via user-provided operation (public member function)
<code>swap</code>	swaps the contents (public member function)

Search

std::string

Конкатенация (сложение) строк: операция “+”

Пример: Даны две строки «Иван» и «Иванов» – мы складываем эти две строки и добавляем пробел между ними.

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    string firstName = "Иван";
    string lastName = "Иванов";
    string fullName = firstName + " " + lastName;
    cout << "Полное имя: " << fullName << endl;
    return 0;
}
```

std::string

Поиск подстроки `find()`

Пример:

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    string text = "Я люблю программировать!";
    string search = "люблю";
    int position = text.find(search);
    if (position != string::npos) {
        cout << "Слово найдено на позиции: " << position << endl;
    } else {
        cout << "Слово не найдено" << endl;
    }
    return 0;
}
```

`text.find(search)` – эта функция ищет подстроку `search` в строке `text` и возвращает ее позицию. Если подстрока не найдена, функция возвращает `string::npos`.

std::string

```
string s1 = "Иван", s2 = "Иванов", s3;
```

- Сравнение:

```
if ( s1 == s2 ){}
```

- Скопировать одну строку в другую можно с помощью обычной операции присваивания:

```
s3 = s2; // копируем s2 в s3
```

std::string

Нумерация символов в строке начинается с нуля. `getline(cin, s);` позволяет прочитать всю строку, включая пробелы. Если бы мы использовали обычный оператор ввода `cin >> s;`, то программа прочитала бы только первое слово до первого пробела. Например, если бы вы ввели строку "Привет, мир!", то `cin` прочитал бы только "Привет,".

Выделение части строки – метод `substr()`

```
s = "0123456789";  
s1 = s.substr ( 3, 5 );  
cout << s1 << endl;
```

Фрагмент копирует в строку `s1` пять символов строки `s` (с 3-го по 7-й).

Этот метод принимает два параметра: номер начального символа и количество символов.

Если второй параметр при вызове `substr` не указан, метод возвращает все символы до конца строки. Например,

```
s = "0123456789";  
s1 = s.substr ( 3 );
```

вернёт «3456789».

Удаление части строки - метод **erase()**

```
s = "0123456789";  
s.erase ( 3, 6 );
```

В строке **s** остаётся значение «0129» (удаляются 6 символов, начиная с 3-го).

Обратите внимание, что процедура **erase** изменяет строку.

Вставка символов в строку – метод `insert()`

```
s = "0123456789";  
s.insert ( 3, "ABC" );
```

Переменная `s` получит значение «012ABC3456789».

Преобразование строки в число

Си-шный способ: `atoi()`

- `std::stoi` – для преобразования строки в целое число.
- `std::stod` – для преобразования строки в число с плавающей точкой: